

Pendientes — Sistema de Gestión Logística y Flota

Documento único de pendientes del proyecto (reemplaza versiones anteriores y duplicadas). Estado al 23/09/2026: ya resuelto y fuera de esta lista — build de producción, preparación para Vercel/Render, archivos en MySQL, selector de fecha único.

■ Deploy (lo que falta para tenerlo online)

- 1 Elegir y crear la base de datos MySQL en internet.** Render no ofrece MySQL gratis. La opción gratuita más directa es Aiven, con 1 GB. Exige conexión cifrada, así que probablemente haya que ajustar un poco cómo se conecta el backend.
- 2 Crear los servicios** siguiendo el README (sección "Deploy"): el backend en Render con `render.yaml` y el frontend en Vercel. Después, cargar el seed de demostración desde la terminal (Shell) del servicio en Render.
- 3 Calibrar TRUST_PROXY** una vez desplegado, con los logs de Render (está explicado en el README).
- 4 Revisar la URL del backend en** `frontend/vercel.json`. Si Render le asigna al servicio un nombre distinto de `gestion-logistica-api`, hay que actualizarla ahí.

■ Falta para cumplir la consigna (aprobación)

- 5 Test de integración del backend contra una base real.** Por ejemplo: crear, asignar y finalizar un viaje con supertest.
- 6 Test E2E automatizado** con Playwright o Cypress. La guía E2E actual es manual.
- 7 Documentación de la API** con Swagger/OpenAPI.
- 8 Video de demostración.**
- 9 Evidencia de ejecución de los tests.** `docs/PLAN-DE-PRUEBAS.md` sigue diciendo "23 / 5 tests"; hoy son 64 en el backend y 36 en el frontend.
- 10 Gestión del proyecto:** falta declarar la metodología, las minutas de reuniones y el tracking de tareas en `docs/`.
- 11 Links a los pull requests en** `proposals.md`. Ya se piden para la regularidad.
- 12 Participación:** cada integrante tiene que tener commits propios y al menos un test de su autoría. Hoy Santiago no tiene commits en el repo oficial.
- 13 Links del deploy y credenciales** para la entrega, cuando esté online.

■ Diferencias entre la propuesta y lo implementado

- 14 Vista de detalle de Vehículo.** La consigna exige un detalle al seleccionar un elemento de cada listado. Choferes, Usuarios y Alertas tampoco tienen vista de detalle.
- 15 "Historial de mantenimientos por vehículo" en la interfaz.** El backend ya acepta filtrar por vehículo; falta el filtro en la pantalla de Mantenimiento.
- 16 Dashboard:** faltan "Kilometraje total por vehículo" y "Alertas abiertas por tipo".
- 17 Reescribir "CRUD Auditoría" y "CRUD Alerta" en la propuesta.** La auditoría no se edita a propósito, y las alertas solo se crean y se resuelven. Así escrito, parece que falta algo.

■ Bugs a corregir

- 18 Se puede eliminar un chofer con un viaje en curso.** Desactivarlo está bloqueado, pero eliminarlo no.
- 19 Carrera en la baja de vehículos y de choferes.** Falta el bloqueo de fila que ya usan viajes y mantenimientos. Sin él, si la baja coincide con una asignación, puede quedar un vehículo inactivo con un viaje en curso.

■ Menores / prolijidad

- 20 Vulnerabilidades de dependencias:** 18 en el backend y 8 en el frontend. Varias se arreglan con actualizaciones menores. Prisma y Vite requieren cambio de versión mayor, y en ningún caso hay que correr `npm audit fix --force`.
- 21 El frontend se carga en un solo archivo de 1,2 MB.** Se puede partir por pantalla con `React.lazy`.
- 22 Documentación desactualizada:** el README dice 57 endpoints y son 60. Algunos capítulos del manual técnico dicen que no hay tests de componentes ni de servicios, y ya los hay.
- 23 Limpiar la raíz del repo:** `Backend-Gestion-Logistica.docx` (la cátedra no acepta .docx) y `CRUD-proposal-ubicacion-codigo.pdf` si ya cumplió su propósito.
- 24 Warning de lint pendiente** en `auth/guards.tsx`.
- 25 Lockfile del frontend:** con `npm 10`, `npm ci` lo marca desincronizado. Conviene regenerarlo con la versión de `npm` que use el equipo.

■ Mejoras de UX pedidas (Román, 23/09/2026)

- 26 Buscador en Viajes (chofer y destino).** Ícono de lupa que filtre por nombre del chofer asignado y por destino. A definir: un campo único o dos separados, y si la búsqueda es server-side (preferible) o client-side.
- 27 Ordenamiento en Alertas y Mantenimiento.** Hoy ambas pantallas solo filtran, no ordenan. Mantenimiento: por tipo, por vehículo, por período. Alertas: por tipo, por vehículo, por período semanal u otro rango. Implica `sortBy/sortOrder` en los endpoints y el control en la UI.
- 28 Reorganización del frontend por componente + código en inglés.** Cambio de fondo: carpeta por componente (`.tsx` / `.styles` / `.types.ts` / `.data.ts` / `.const.ts` / `.helpers.ts` / `.api.ts`, según referencia adjunta) y pasar identificadores, comentarios y nombres de archivo a inglés. A discutir en equipo antes de arrancar: si se adopta el patrón de referencia tal cual (SCSS, callbacks `onSuccess/onError`) o adaptado a MUI/axios; y si "todo en inglés" incluye los textos que ve el usuario o solo el código.
- 29 Evaluación de alertas: automática una vez al día + botón manual siempre disponible.** Hoy corre cada 10 minutos (configurable). Pasarlo a una vez al día; el botón "Evaluar alertas" queda igual para uso manual en cualquier momento.
- 30 Carteles aclaratorios cuando una acción no se puede hacer.** Reforzar los mensajes visibles al usuario en todo el sistema (reglas de negocio bloqueadas, permisos, validaciones), no solo en login. Definir si alcanza el Alert actual o hace falta algo más visible (Snackbar/Toast).

Prioridad sugerida (primera entrega: 12 al 16 de octubre): primero el deploy (1 a 4), después los tests de integración y E2E junto con la documentación de la API (5 a 7), y después la vista de detalle (14), por ser un requisito explícito de la consigna. Los puntos 26 a 30 son mejoras de UX sin fecha límite de la cátedra.